

REPORTE DE PRÁCTICA NO. 7

Análisis sintáctico de elementos de un lenguaje de programación

ALUMNO:

Contreras Hernandez Diego Adonay
Martinez Delgado Baruchs Jesua
Vite Juares Alberick
Navarrete Torres Oscar

PROFESOR: Dr. Eduardo Cornejo-Velázquez



1. Introducción

El diseño e implementación de compiladores es una de las áreas más fundamentales y complejas dentro de las ciencias de la computación. El proceso de compilación se divide tradicionalmente en dos etapas principales: el front-end (análisis) y el back-end (síntesis). Dentro de la fase de análisis, el análisis sintáctico desempeña un papel crucial al recibir la secuencia de componentes léxicos (tokens) generados por el analizador léxico y verificar que dicha secuencia cumpla con las reglas gramaticales del lenguaje de programación (Aho, Sethi y Ullman, 1998). En lenguajes basados en gramáticas formales, el análisis sintáctico resulta esencial para facilitar la comprensión de la estructura del código fuente y abordar de manera efectiva errores difíciles de identificar, como la ambigüedad en la formulación de ciertas sentencias. Esta práctica se centra en la construcción de un analizador sintáctico para una de las estructuras de control de flujo más importantes: el condicional if-then. Utilizando herramientas estándar de la industria como Lex (para el análisis léxico) y Yacc (para el análisis sintáctico), se demostrará cómo una gramática de contexto libre puede ser automatizada para validar estructuras complejas de manera eficiente.

2. Marco teórico

El desarrollo de un traductor o compilador se divide en varias fases secuenciales. Esta práctica se centra en las dos primeras: el análisis léxico y el análisis sintáctico.

1. Análisis Léxico (Escaneo) Es la primera fase de un compilador. lee el flujo de caracteres de entrada y los agrupa en unidades con significado propio llamadas tokens.

Tokens: Son las categorías gramaticales del lenguaje (SUMA, ID, ENTERO).

Lexema: Es la secuencia de caracteres real que coinciden con un token ("3.1416" es el lexema para el token DECIMAL).

Autómatas Finitos Deterministas (AFD): Son modelos matemáticos que sirven para reconocer lenguajes regulares. Flex utiliza estos autómatas para identificar patrones definidos mediante expresiones regulares.

2. Análisis Sintáctico (Parsing) Esta fase recibe los tokens generados por el analizador léxico y determina si la estructura de la oración u operación es válida según las reglas gramaticales.

Gramáticas Libres de Contexto (GLC): Son un conjunto de reglas que definen cómo se pueden combinar los tokens. Se representan comúnmente mediante la notación BNF (Backus-Naur Form).

Jerarquía de Operadores: Para que una calculadora funcione correctamente, el analizador debe aplicar reglas de precedencia. En este proyecto, se implementa una estructura jerárquica donde la multiplicación y división tienen mayor prioridad que la suma y resta.

Árbol de Análisis Sintáctico: Es una representación jerárquica que muestra cómo una cadena de entrada se deriva de las reglas de la gramática.

4. Manejo de Errores Un compilador robusto debe ser capaz de identificar y reportar errores en dos niveles:
Error Léxico: Ocurre cuando el escáner encuentra un carácter que no pertenece a ningún token válido (ej. un símbolo @ en una expresión aritmética).

Error Sintáctico: Ocurre cuando los tokens son válidos pero su orden no respeta las reglas de la gramática (ej. $5 + * 3$).

3. Herramientas empleadas

(Flex y Bison) Flex (Fast Lexical Analyzer): Es una herramienta para generar escáneres. Su entrada es un archivo .l que contiene expresiones regulares y su salida es código fuente en C que implementa el autómata.

Bison (Yacc compatible): Es un generador de analizadores sintácticos de propósito general. Convierte una gramática libre de contexto en un analizador LALR (Look-Ahead Left-to-Right). Utiliza una pila para procesar los tokens y realizar reducciones gramaticales.

4. Objetivos

general

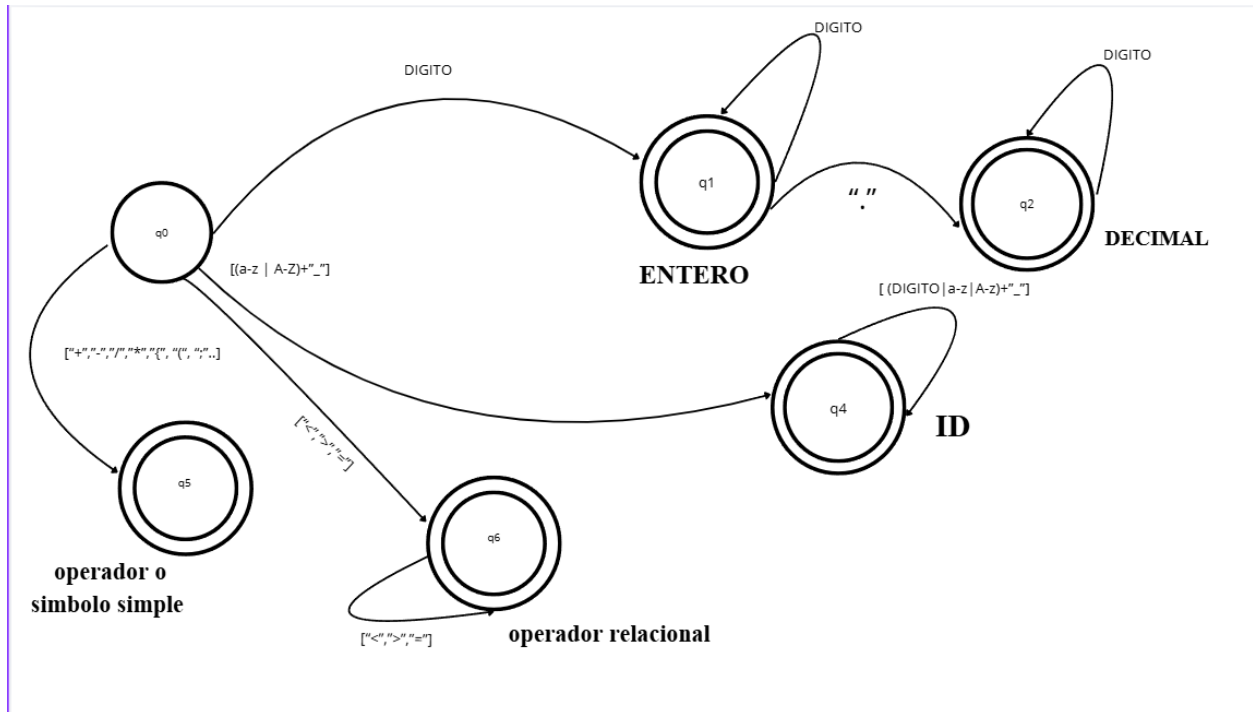
Implementar un analizador léxico y un analizador sintáctico utilizando las herramientas Lex y Yacc para reconocer, validar y estructurar la sentencia condicional if-then, logrando la detección automatizada de errores y la validación de gramáticas propias de un lenguaje de programación de alto nivel.

Específicos

- Emplear herramientas conceptuales derivadas de la teoría de autómatas y lenguajes formales para definir el comportamiento del analizador.
- Diseñar un programa en Lex que identifique de forma unívoca los tokens asociados a palabras clave (if, then), expresiones condicionales (operadores relacionales) y bloques de código (identificadores y números).
- Implementar un programa en Yacc que aplique las reglas de producción para verificar la correcta construcción de la estructura if...then, generando acciones de aceptación o rechazo.

4. Desarrollo

Esquema de los AF



Notacion BNF

```
<programa> ::= <lista_instrucciones>

<lista_instrucciones> ::= <instruccion>
                        | <lista_instrucciones> <instruccion>

<instruccion> ::= <expresion> ";"
                | <asignacion> ";"
                | <estructura_if>
                | <estructura_while>
                | <estructura_for>
                | <estructura_switch>

<bloque> ::= <instruccion>
           | "{" <lista_instrucciones> "}"

<asignacion> ::= ID "=" <expresion>

<estructura_if> ::= "if" "(" <condicion> ")" "then" <bloque>

<estructura_while> ::= "while" "(" <condicion> ")" <bloque>

<estructura_for> ::= "for" "(" <asignacion> ";" <condicion> ";" <asignacion> ")" <bloque>

<estructura_switch> ::= "switch" "(" ID ")" "{" <lista_casos> "}"

<lista_casos> ::= <caso>
                | <lista_casos> <caso>

<caso> ::= "case" ENTERO ":" <lista_instrucciones>
          | "default" ":" <lista_instrucciones>

<condicion> ::= <expresion> <operador_rel> <expresion>

<operador_rel> ::= ">" | "<" | ">=" | "<=" | "==" | "!="

<expresion> ::= <expresion> "+" <termino>
              | <expresion> "-" <termino>
              | <termino>

<termino> ::= <termino> "*" <factor>
            | <termino> "/" <factor>
            | <factor>

<factor> ::= "(" <expresion> ")"
           | ENTERO
           | DECIMAL
           | ID
```

Código flex

```
%{
    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>
    #include "parser2.tab.h"
}%}

%{
    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>
    #include "parser2.tab.h"
}%}

%option yylineno

DIGITO [0-9]
ID      [a-zA-Z][a-zA-Z0-9]*

%%
"if"      { return IF; }
"then"    { return THEN; }
"while"   { return WHILE; }
"for"     { return FOR; }
"switch"  { return SWITCH; }
"case"    { return CASE; }
"default" { return DEFAULT; }

{DIGITO}+ { yylval.num = atof(yytext); return ENTERO;}

{DIGITO}+"."{DIGITO}+ { yylval.num = atof(yytext); return DECIMAL;}

{ID} { yylval.id = strdup(yytext); return ID;}

">="      { return MAI; }
"<="      { return MEI; }
"=="      { return EQ; }
"!="      { return DIF; }
">"       { return MAQ; }
"<"       { return MEQ; }

"+"       { return SUMA; }
"_"       { return RESTA; }
"*"       { return MULTI; }
"/"       { return DIVI; }
"="       { return ASIG; }

"{"       { return L_CORCHETE; }
"}"       { return R_CORCHETE; }
":"       { return COLON; }
"("       { return P_I; }
")"       { return P_D; }
```

```
";"          { return P_COMA; }

[ \t\r\n]    {  }

. {
    printf("Error léxico en línea %d: Carácter no reconocido '%s'\n", yylineno, yytext);
}

%%

int yywrap(void) {
    return 1;
}
```

Código Bison

```
%{
    #include <stdio.h>
    #include <stdlib.h>

    extern int yylex();
    extern int yylineno;
    extern char *yytext;
    extern FILE *yyin;

    void yyerror(const char *s);
}%}

%union {
    float num;
    char *id;
}

%token <num> ENTERO DECIMAL
%token <id> ID
%token IF THEN WHILE FOR SWITCH CASE DEFAULT
%token SUMA RESTA MULTI DIVI ASIG
%token P_I P_D P_COMA DIF MAQ MEQ MAI MEI EQ
%token L_CORCHETE R_CORCHETE COLON

%%

programa:
    lista_instrucciones
    ;

lista_instrucciones:
    instruccion
    | lista_instrucciones instruccion
    ;

instruccion:
    expresion P_COMA          { printf("Línea %d: Expresión matemática correcta.\n", yylineno); }
    | asignacion P_COMA      { printf("Línea %d: Asignación de variable correcta.\n", yylineno); }
    | estructura_if          { printf("Línea %d: Estructura IF correcta.\n", yylineno); }
    | estructura_while       { printf("Línea %d: Estructura WHILE correcta.\n", yylineno); }
    | estructura_for         { printf("Línea %d: Estructura FOR correcta.\n", yylineno); }
    | estructura_switch      { printf("Línea %d: Estructura SWITCH correcta.\n", yylineno); }
    | error P_COMA          { yyerrok; }
    ;

asignacion:
    ID ASIG expresion
    ;

estructura_if:
    IF P_I condicion P_D THEN bloque
    ;
```



```

estructura_while:
    WHILE P_I condicion P_D bloque
    ;

estructura_for:
    FOR P_I asignacion P_COMA condicion P_COMA asignacion P_D bloque
    ;

estructura_switch:
    SWITCH P_I ID P_D L_CORCHETE lista_casos R_CORCHETE
    ;

lista_casos:
    caso
    | lista_casos caso
    ;

caso:
    CASE ENTERO COLON lista_instrucciones
    | DEFAULT COLON lista_instrucciones
    ;

condicion:
    expresion operador_rel expresion
    ;

operador_rel:
    MAQ | MEQ | MAI | MEI | EQ | DIF
    ;

bloque:
    instruccion
    | L_CORCHETE lista_instrucciones R_CORCHETE
    ;

expresion:
    expresion SUMA termino
    | expresion RESTA termino
    | termino
    ;

termino:
    termino MULTI factor
    | termino DIVI factor
    | factor
    ;

factor:
    P_I expresion P_D
    | ENTERO
    | DECIMAL
    | ID
    ;

```

```

%%

void yyerror(const char *s) {
    fprintf(stderr, "Error sintáctico en línea %d, cerca de '%s': %s\n", yylineno, yytext, s);
}

int main(int argc, char **argv) {
    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            printf("Error: No se pudo abrir el archivo %s\n", argv[1]);
            return 1;
        }
        printf("--- Analizando el archivo: %s ---\n", argv[1]);
    } else {
        printf("Uso correcto: %s <archivo.txt>\n", argv[0]);
        return 1;
    }
    yyparse();

    fclose(yyin);
    printf("--- Análisis finalizado ---\n");
    return 0;
}

```

Pruebas

El objetivo principal de este programa es procesar un archivo de texto que contiene código fuente y verificar si está correctamente escrito, basándose en un conjunto de reglas gramaticales previamente definidas. Para lograr esto el funcionamiento del sistema se dividió en dos etapas: la identificación de palabras y símbolos ; y la validación de la estructura del código.

En la primera etapa, el programa revisa el documento carácter por carácter para agruparlos en tokens. Específicamente, el programa está diseñado para reconocer las siguientes palabras reservadas:

- `if` | `then` |
- `while` |
- `for` |
- `switch` |, `case` | y `default` | \ \

A la par de estas palabras clave, el programa también es capaz de distinguir identificadores (nombres de variables), números enteros o decimales, operadores matemáticos y los símbolos de comparación necesarios para evaluar las condiciones lógicas (como `<`, `>=`, `==`, etc.).

Una vez que se reconocen todos los elementos de una instrucción, entra en acción la segunda etapa, donde el programa verifica si el orden en el que se escribieron tiene sentido lógico y obedece a las reglas del lenguaje.

Resultados que arroja el programa:

Como resultado final, el analizador imprime en la consola un diagnóstico detallado, evaluando el archivo línea por línea.

- **En caso de éxito:** El programa arroja mensajes de validación que confirman la estructura detectada. Por ejemplo, imprime mensajes como *"Línea 15: Estructura IF correcta"* o *"Línea 8: Asignación de variable correcta"*.
- **En caso de fallo:** Arroja una alerta de error sintáctico o léxico. Este mensaje indica la línea exacta donde el análisis falló y muestra cerca de qué texto o símbolo se generó el conflicto, lo cual facilita enormemente la tarea de depuración.

para la compilación del código se siguieron los siguientes pasos desde la terminal cmd de la computadora. Se obtienen 3 archivos `"lexer2.l"` para el analizador léxico `"parser2.y"` para el analizador sintáctico. primero se determina la dirección en la que están los archivos `.l` y `.y` para su compilación; y desde el cmd se ejecutan los siguientes comandos.

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>flex lexer2.l
```

- esta parte crea el archivo `lex.yy.c` que contiene un `include "parser2.tab.h"` para vincularse con el otro analizador sintáctico

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>bison -d parser2.y
```

-esta parte crea los archivos `parser.tab.y` y `parser2.tab.h` para su ejecución

-como último paso se genera el `.exe` de la ejecución del programa con el comando:

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>gcc parser2.tab.c lex.yy.c -o "nombre archivo".exe
```

Se ingresa un archivo sin errores sintacticos ni lexicos por medio de un archivo .txt desde la consola con el comando:

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>"nombre archivo".exe<entradas_correctas.txt
```

y se obtienen los siguientes resultados:

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>mi_compilador.exe P_correcto.txt
--- Analizando el archivo: P_correcto.txt ---
LÝnea 1: Asignacin de variable correcta.
LÝnea 2: Asignacin de variable correcta.
LÝnea 3: Asignacin de variable correcta.
LÝnea 4: Asignacin de variable correcta.
LÝnea 7: Asignacin de variable correcta.
LÝnea 8: Asignacin de variable correcta.
LÝnea 10: Asignacin de variable correcta.
LÝnea 11: Estructura IF correcta.
LÝnea 12: Estructura IF correcta.
LÝnea 15: Asignacin de variable correcta.
LÝnea 16: Asignacin de variable correcta.
LÝnea 17: Asignacin de variable correcta.
LÝnea 19: Asignacin de variable correcta.
LÝnea 20: Estructura IF correcta.
LÝnea 21: Estructura WHILE correcta.
LÝnea 24: Asignacin de variable correcta.
LÝnea 26: Asignacin de variable correcta.
LÝnea 27: Asignacin de variable correcta.
LÝnea 28: Estructura WHILE correcta.
LÝnea 29: Estructura FOR correcta.
LÝnea 33: Asignacin de variable correcta.
LÝnea 35: Asignacin de variable correcta.
LÝnea 36: Asignacin de variable correcta.
LÝnea 38: Asignacin de variable correcta.
LÝnea 39: Asignacin de variable correcta.
LÝnea 40: Estructura SWITCH correcta.
LÝnea 42: Asignacin de variable correcta.
```

por ultimo se compila con el archivo con errores en su codigo con el mismo comando llamando a un archivo .text desde la consola

C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>"nombre_archivo".exe<entradas_incorrectas.txt

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>mi_compilador.exe p_errores.txt
--- Analizando el archivo: p_errores.txt ---
Línea 1: Asignaci n de variable correcta.
Línea 2: Asignaci n de variable correcta.
Error sint ctico en L nea 6, cerca de 'resultado': syntax error
Error L xico en L nea 6: Car cter no reconocido '&'
Error sint ctico en L nea 8, cerca de '{': syntax error
L nea 10: Asignaci n de variable correcta.
Error sint ctico en L nea 11, cerca de '}': syntax error
Error sint ctico en L nea 15, cerca de ';': syntax error
L nea 16: Asignaci n de variable correcta.
Error sint ctico en L nea 17, cerca de '}': syntax error
Error L xico en L nea 19: Car cter no reconocido ','
Error L xico en L nea 19: Car cter no reconocido ','
Error sint ctico en L nea 21, cerca de '}': syntax error
Error sint ctico en L nea 26, cerca de 'case': syntax error
Error sint ctico en L nea 28, cerca de 'default': syntax error
Error sint ctico en L nea 30, cerca de '}': syntax error
L nea 33: Asignaci n de variable correcta.
Error sint ctico en L nea 35, cerca de '{': syntax error
Error sint ctico en L nea 37, cerca de '}': syntax error
Error sint ctico en L nea 41, cerca de '}': syntax error
Error L xico en L nea 43: Car cter no reconocido '-'
Error L xico en L nea 43: Car cter no reconocido '@'
L nea 47: Asignaci n de variable correcta.
Error sint ctico en L nea 48, cerca de 'variable': syntax error
Error sint ctico en L nea 50, cerca de 'default': syntax error
Error sint ctico en L nea 52, cerca de '}': syntax error
L nea 57: Asignaci n de variable correcta.
Error sint ctico en L nea 58, cerca de ';': syntax error
```

5. Conclusiones

La elaboración de esta práctica demostró de manera práctica la distinción fundamental entre la validez de los componentes léxicos y la coherencia estructural de los mismos. El uso de gramáticas de contexto libre, implementadas a través de Yacc y alimentadas por Lex, permite aislar la lógica matemática del lenguaje de programación de la complejidad de leer cadenas de texto. Se concluye que un analizador sintáctico no solo verifica la legalidad del código, sino que también impone un orden jerárquico que es el paso previo e indispensable para la generación de árboles de sintaxis abstracta (AST) y la posterior generación de código máquina. La detección inmediata del error por omisión del token THEN evidencia cómo las reglas de producción actúan como un escudo robusto contra ambigüedades en lenguajes de alto nivel.

Referencias Bibliográficas

Carranza Sahagún, D. U. (Coord.). (2024).

Compiladores: Fases de análisis. Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). Lenguajes formales y teoría de autómatas. Editorial Alfaomega. Argentina.

Gutú, O. (2013). Primer curso en teoría de autómatas y lenguajes formales. Pearson Educación. México.

Hopcroft, J. E. Motwani, R. Ullman, J. D. (2007). Introducción a la teoría de automatas, lenguajes y computación. Pearson Educación. México.